

# 双交点图册 / 先看输入是否合法

一个正方形面用 4 个角点位 + 4 个 pair 位编码。每次穿越都会翻转内外，两次穿越后回到原符号。

key = corners | (pairs << 4)：低四位记录角点内外，高四位记录各边是否请求两个交点。

| 边两端符号 | pair = 0 | pair = 1（请求两次穿越） |
|-------|----------|------------------|
| 同号    | 0 个交点：合法 | 2 个交点：合法         |
| 异号    | 1 个交点：合法 | 非法：两次翻转无法接到异号终点  |

## 第一章 合法输入

82 个合法编码经面内旋转、镜像归并为 23 个代表模板。按内部角点数排列，展示每个模板的一种立方体延伸。

02-05 页

## 第二章 非法输入

174 个输入按矛盾边数分成 104、60、8、2 个。先看四张分步解释图，再查全部编码矩阵；它们没有可绘制的合法曲面。

06-18 页

## 附录 合法输入的内外翻转连接差异

7 组对照的两侧都属于合法输入。连接选择不同，不计入 174 个非法编码。

19-20 页

图例：大实心角点=内，大空心角点=外，小圆=边交点，黄色=示意转折点；红色只标非法输入中的矛盾边。合法图中的不同线色区分闭环。

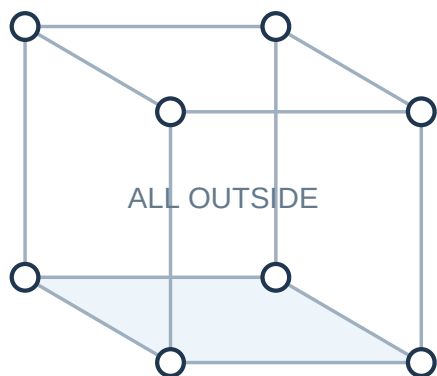
合法立方体：同一面输入复制到  $z=0$  和  $z=1$ ，竖边无交点。几何位置仅作示意，不代表真实网格重建。

## 合法模板 / 0 个内部角点

0 内 / 4 外

**CASE 1a**

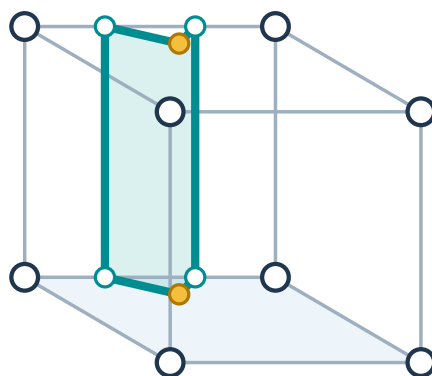
0 loops



face 0x00 counts [0, 0, 0, 0]  
cube 0x00000

**CASE 1b**

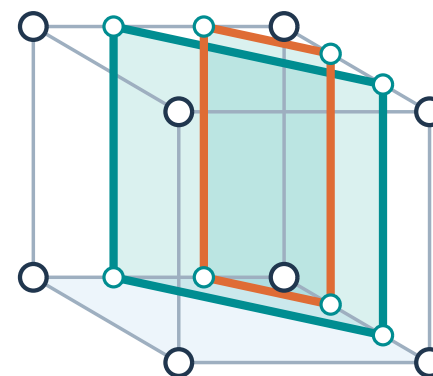
1 loops



face 0x10 counts [2, 0, 0, 0]  
cube 0x00500

**CASE 1c**

2 loops

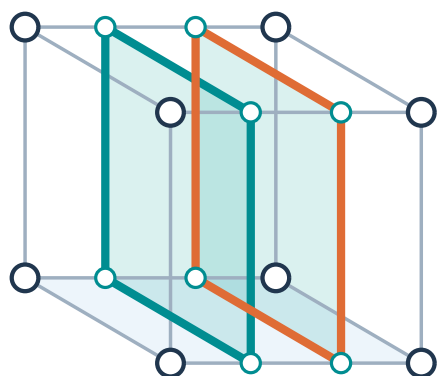


face 0x30 counts [2, 2, 0, 0]  
cube 0x0A500

0 内 / 4 外 (续)

**CASE 1d**

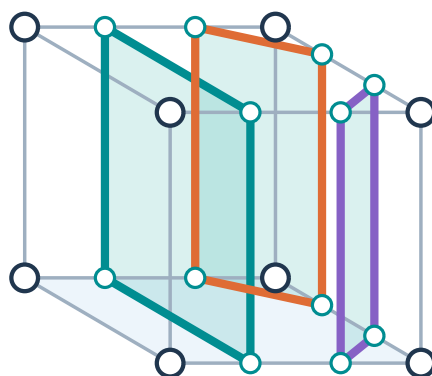
2 loops



face 0x50 counts [2, 0, 2, 0]  
cube 0x00F00

**CASE 1e**

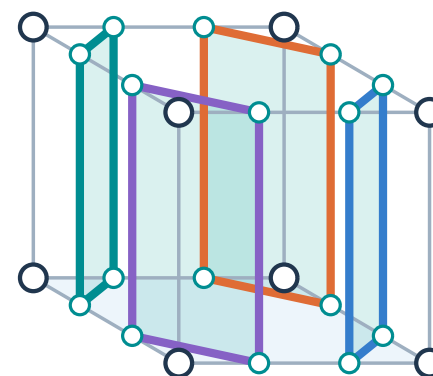
3 loops



face 0x70 counts [2, 2, 2, 0]  
cube 0x0AF00

**CASE 1f**

4 loops



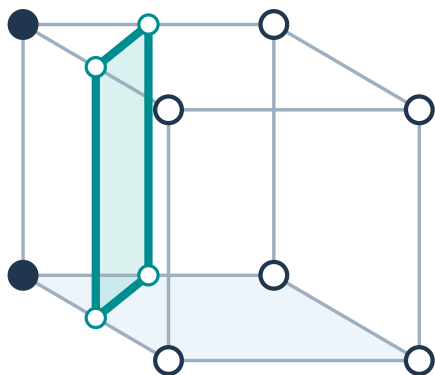
face 0xF0 counts [2, 2, 2, 2]  
cube 0x0FF00

# 合法模板 / 1 个与 3 个内部角点

1 内 / 3 外

**CASE 2a**

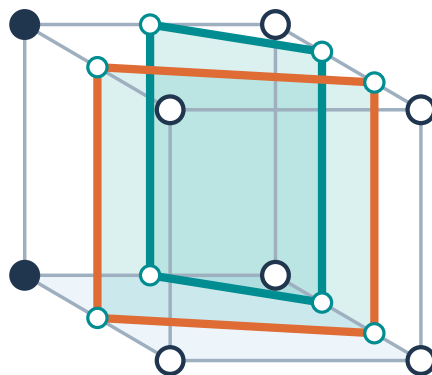
1 loops



face 0x01 counts [1, 0, 0, 1]  
cube 0x00011

**CASE 2b**

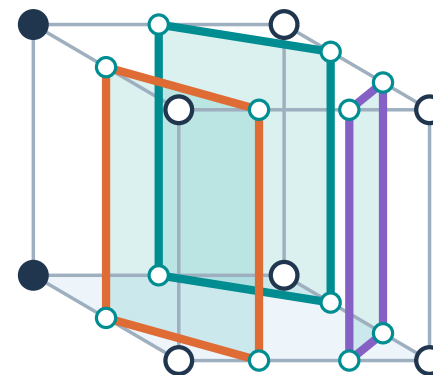
2 loops



face 0x21 counts [1, 2, 0, 1]  
cube 0x0A011

**CASE 2c**

3 loops

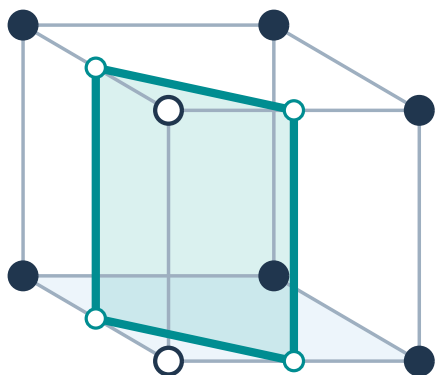


face 0x61 counts [1, 2, 2, 1]  
cube 0x0AA11

3 内 / 1 外

**CASE 11a**

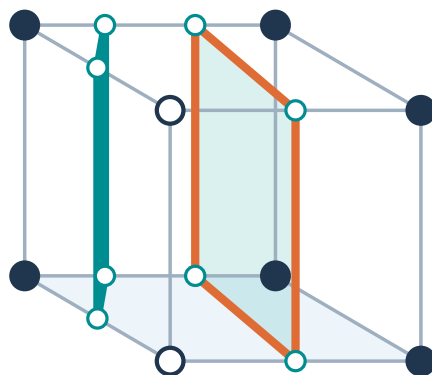
1 loops



face 0x07 counts [0, 0, 1, 1]  
cube 0x000BB

**CASE 11b**

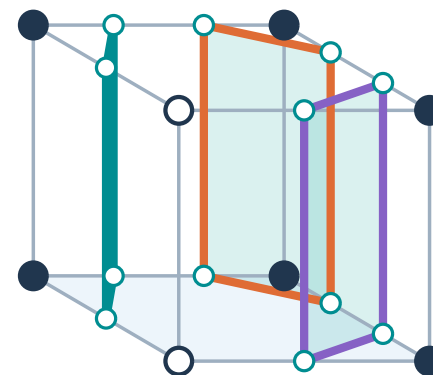
2 loops



face 0x17 counts [2, 0, 1, 1]  
cube 0x005BB

**CASE 11c**

3 loops



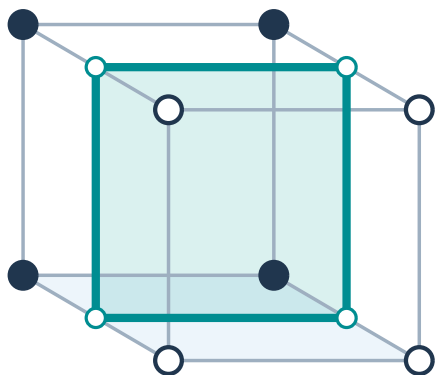
face 0x37 counts [2, 2, 1, 1]  
cube 0x0A5BB

# 合法模板 / 2 个内部角点

内部角点相邻

## CASE 3a

1 loops

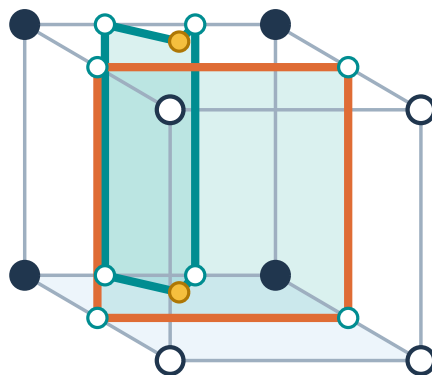


face 0x03 counts [0, 1, 0, 1]

cube 0x00033

## CASE 3b

2 loops

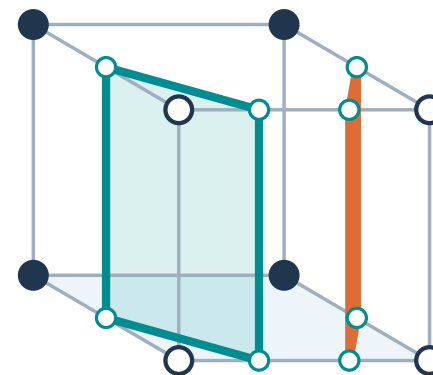


face 0x13 counts [2, 1, 0, 1]

cube 0x00533

## CASE 3c

2 loops



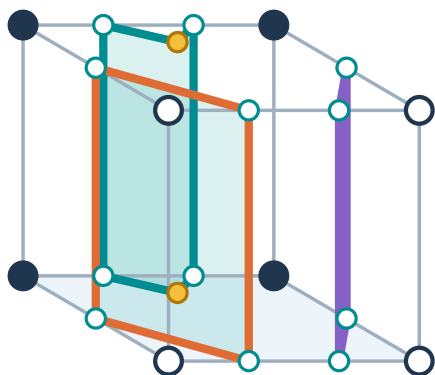
face 0x43 counts [0, 1, 2, 1]

cube 0x00A33

内部角点相邻 (3d) / 对角 (4a)

## CASE 3d

3 loops

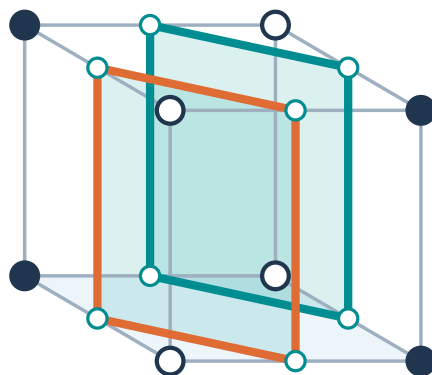


face 0x53 counts [2, 1, 2, 1]

cube 0x00F33

## CASE 4a

2 loops



face 0x05 counts [1, 1, 1, 1]

cube 0x00099

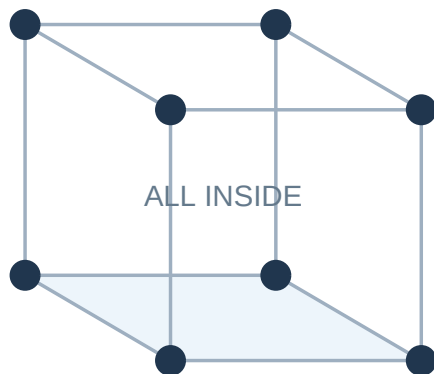
按内部角点数分组后，再按角点相邻或对角、双交点边的位置区分模板。每张图都按源码六面查表结果绘制。

## 合法模板 / 4 个内部角点

4 内 / 0 外

CASE 12a

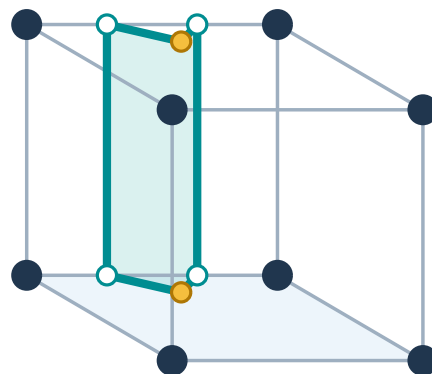
0 loops



face 0x0F counts [0, 0, 0, 0]  
cube 0x00FF

CASE 12b

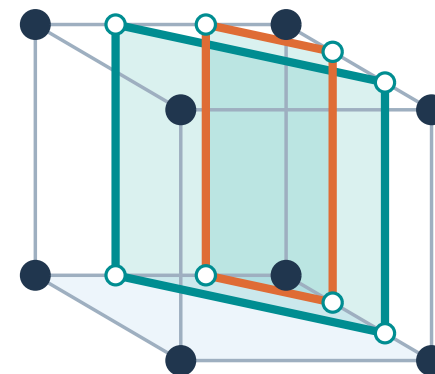
1 loops



face 0x1F counts [2, 0, 0, 0]  
cube 0x005FF

CASE 12c

2 loops

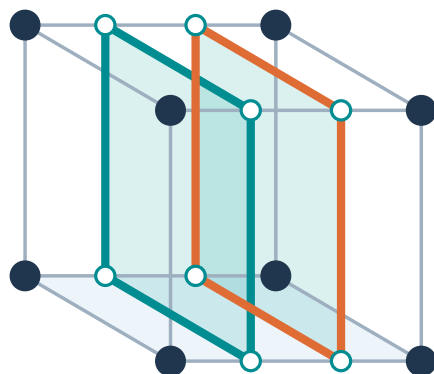


face 0x3F counts [2, 2, 0, 0]  
cube 0x0A5FF

4 内 / 0 外 (续)

CASE 12d

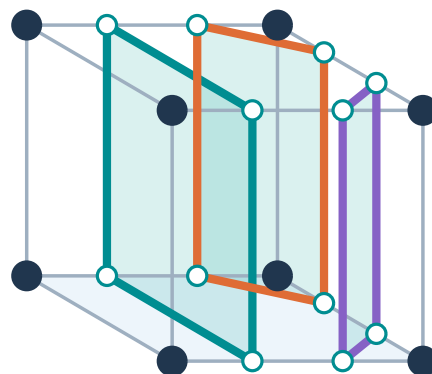
2 loops



face 0x5F counts [2, 0, 2, 0]  
cube 0x00FFF

CASE 12e

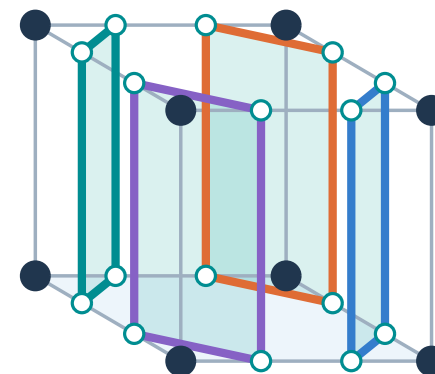
3 loops



face 0x7F counts [2, 2, 2, 0]  
cube 0x0AFFF

CASE 12f

4 loops



face 0xFF counts [2, 2, 2, 2]  
cube 0x0FFFF

# 非法输入 / 1 条矛盾边，共 104 个编码

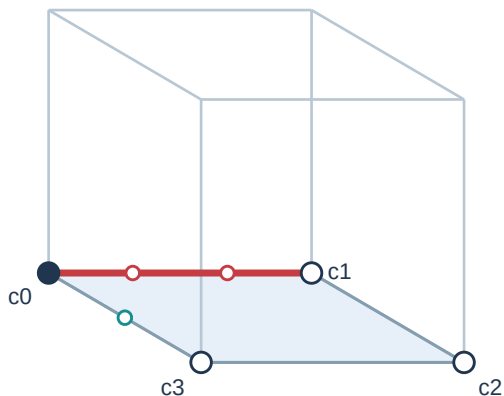
先看线框中是哪一个面，再看面上哪些边自相矛盾，最后沿一条红边数两次穿越。

## 1 条矛盾边 / 104 个编码

代表输入 0x11

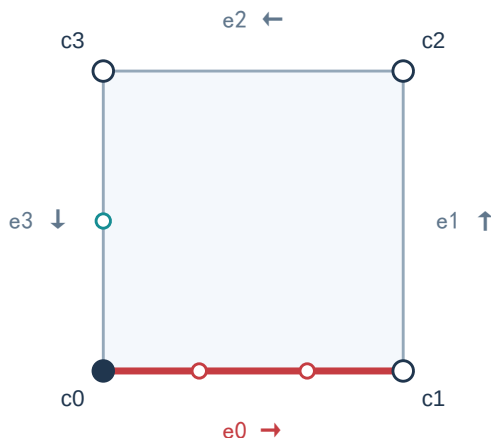
矛盾条件：端点异号，但 pair bit=1 要求两次穿越。每次穿越都会翻转内外。

### ① 立方体中的位置



仅标  $z=0$  面的输入  
其余顶点、边：未指定  
立方体线框只用于定位

### ② 面输入与边方向



矛盾边：e0  
请求交点数：[2, 0, 0, 1]

### ③ 放大 e0: $c0 \rightarrow c1$

沿箭头方向走，两次穿越后回到原符号：



但编码给定的终点是：○ 外

应当为内，却写成外 → 输入自相矛盾

此输入不生成连接线、闭环或曲面。

API: `std::invalid_argument`

实心角点=内；空心角点=外；小圆=请求交点；红色=矛盾边。局部  $c0, c1, c2, c3$  对应立方体顶点 0, 1, 3, 2。

只要有一条矛盾边，整个面输入就非法。多条矛盾边是同一条规则被重复违反；公开接口会抛出 `std::invalid_argument`。

# 非法输入 / 2 条矛盾边，共 60 个编码

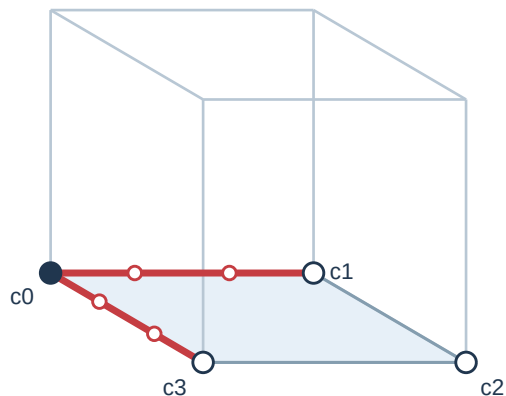
先看线框中是哪一个面，再看面上哪些边自相矛盾，最后沿一条红边数两次穿越。

## 2 条矛盾边 / 60 个编码

代表输入 0x91

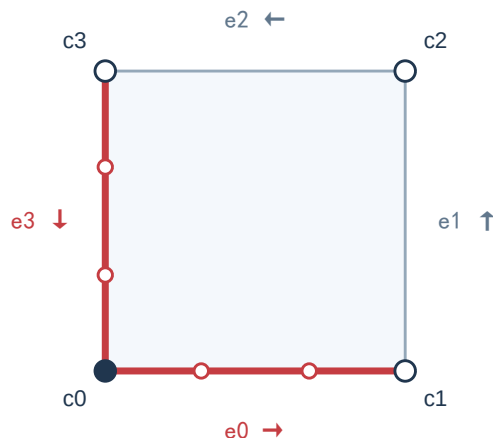
矛盾条件：端点异号，但 pair bit=1 要求两次穿越。每次穿越都会翻转内外。

### ① 立方体中的位置



仅标  $z=0$  面的输入  
其余顶点、边：未指定  
立方体线框只用于定位

### ② 面输入与边方向



矛盾边：e0, e3  
请求交点数：[2, 0, 0, 2]

### ③ 放大 e0: $c0 \rightarrow c1$

沿箭头方向走，两次穿越后回到原符号：



但编码给定的终点是：○ 外

应当为内，却写成外 → 输入自相矛盾

此输入不生成连接线、闭环或曲面。

API: `std::invalid_argument`

实心角点=内；空心角点=外；小圆=请求交点；红色=矛盾边。局部  $c0, c1, c2, c3$  对应立方体顶点 0, 1, 3, 2。

只要有一条矛盾边，整个面输入就非法。多条矛盾边是同一条规则被重复违反；公开接口会抛出 `std::invalid_argument`。

# 非法输入 / 3 条矛盾边，共 8 个编码

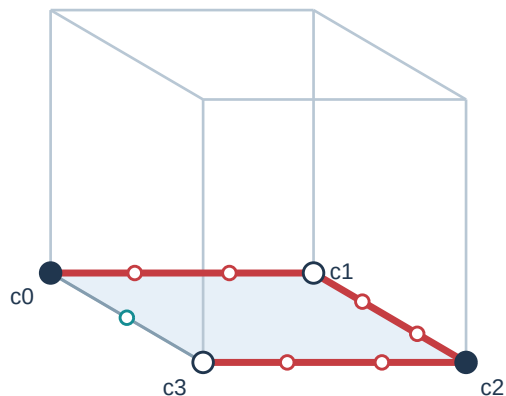
先看线框中是哪一个面，再看面上哪些边自相矛盾，最后沿一条红边数两次穿越。

## 3 条矛盾边 / 8 个编码

代表输入 0x75

矛盾条件：端点异号，但 pair bit=1 要求两次穿越。每次穿越都会翻转内外。

### ① 立方体中的位置

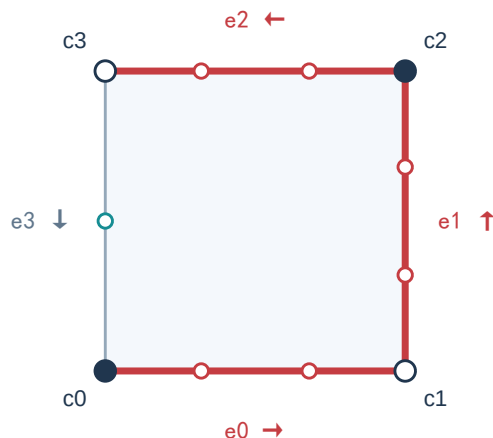


仅标  $z=0$  面的输入

其余顶点、边：未指定

立方体线框只用于定位

### ② 面输入与边方向



矛盾边：e0, e1, e2

请求交点数：[2, 2, 2, 1]

### ③ 放大 e0: $c0 \rightarrow c1$

沿箭头方向走，两次穿越后回到原符号：



但编码给定的终点是：

○ 外

应当为内，却写成外 → 输入自相矛盾

此输入不生成连接线、闭环或曲面。

API: `std::invalid_argument`

实心角点=内；空心角点=外；小圆=请求交点；红色=矛盾边。局部  $c0, c1, c2, c3$  对应立方体顶点 0, 1, 3, 2。

只要有一条矛盾边，整个面输入就非法。多条矛盾边是同一条规则被重复违反；公开接口会抛出 `std::invalid_argument`。

# 非法输入 / 4 条矛盾边，共 2 个编码

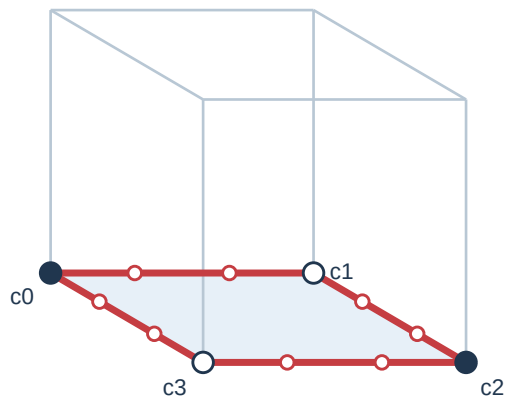
先看线框中是哪一个面，再看面上哪些边自相矛盾，最后沿一条红边数两次穿越。

## 4 条矛盾边 / 2 个编码

代表输入 0xF5

矛盾条件：端点异号，但 pair bit=1 要求两次穿越。每次穿越都会翻转内外。

### ① 立方体中的位置

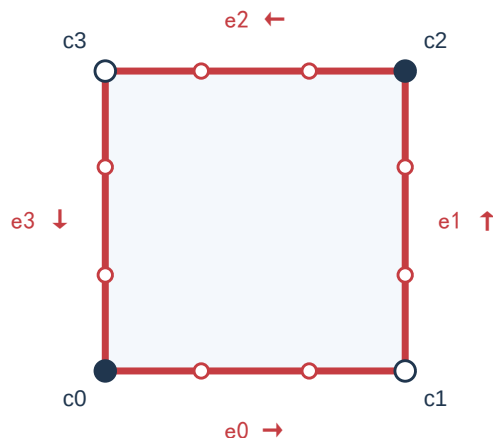


仅标  $z=0$  面的输入

其余顶点、边：未指定

立方体线框只用于定位

### ② 面输入与边方向



矛盾边：e0, e1, e2, e3

请求交点数：[2, 2, 2, 2]

### ③ 放大 e0: $c0 \rightarrow c1$

沿箭头方向走，两次穿越后回到原符号：



但编码给定的终点是：

○ 外

应当为内，却写成外 → 输入自相矛盾

此输入不生成连接线、闭环或曲面。

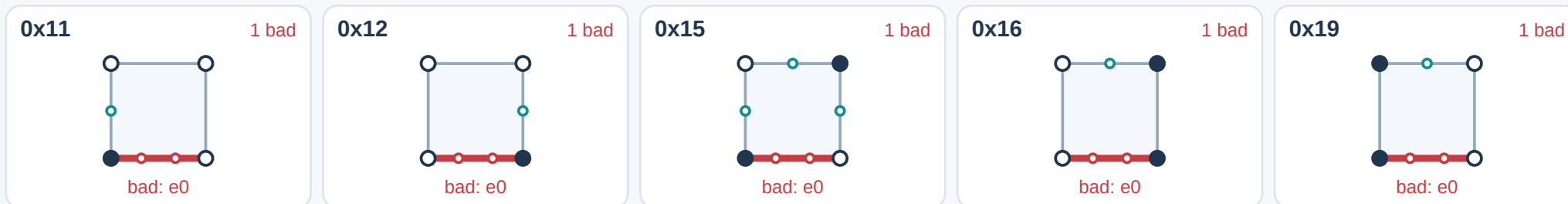
API: `std::invalid_argument`

实心角点=内；空心角点=外；小圆=请求交点；红色=矛盾边。局部  $c0, c1, c2, c3$  对应立方体顶点 0, 1, 3, 2。

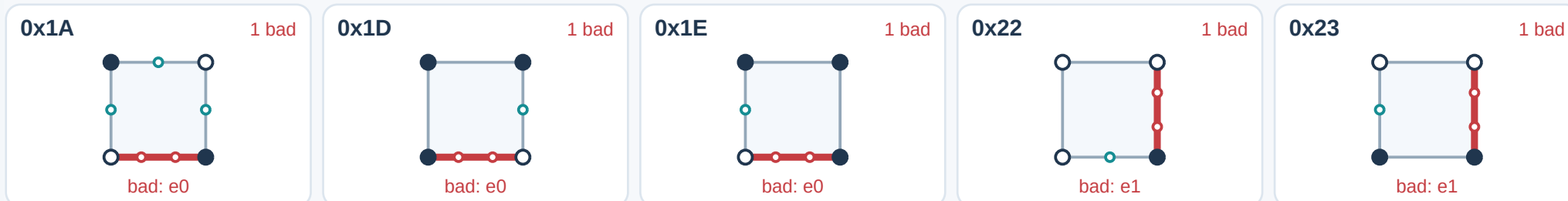
只要有一条矛盾边，整个面输入就非法。多条矛盾边是同一条规则被重复违反；公开接口会抛出 `std::invalid_argument`。

# 非法编码索引 / 1 条矛盾边

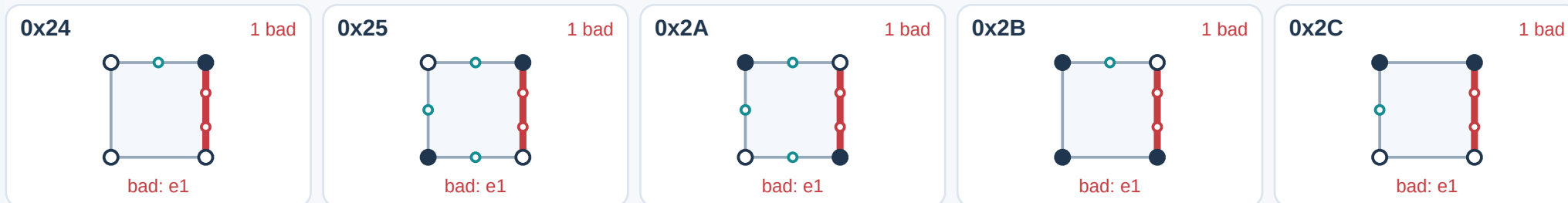
1 条矛盾边 / 本类 104 个 / 第 1-5 个



1 条矛盾边 / 本类 104 个 / 第 6-10 个



1 条矛盾边 / 本类 104 个 / 第 11-15 个

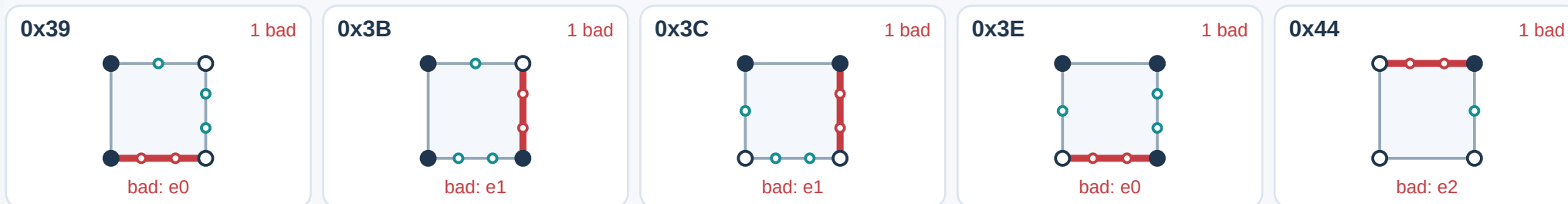


1 条矛盾边 / 本类 104 个 / 第 16-20 个

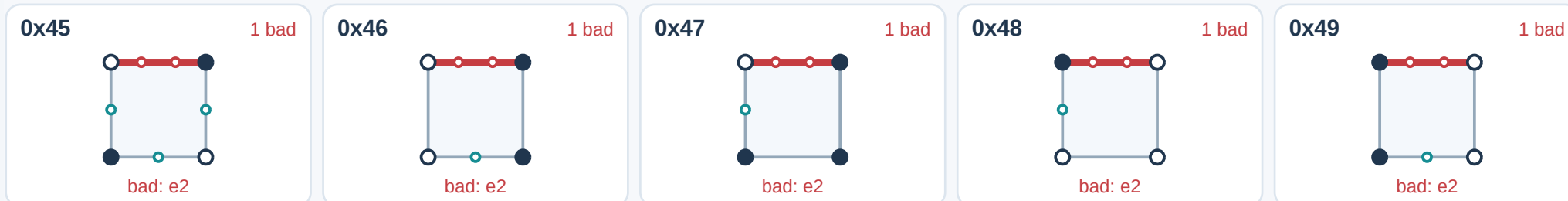


# 非法编码索引 / 1 条矛盾边

1 条矛盾边 / 本类 104 个 / 第 21-25 个



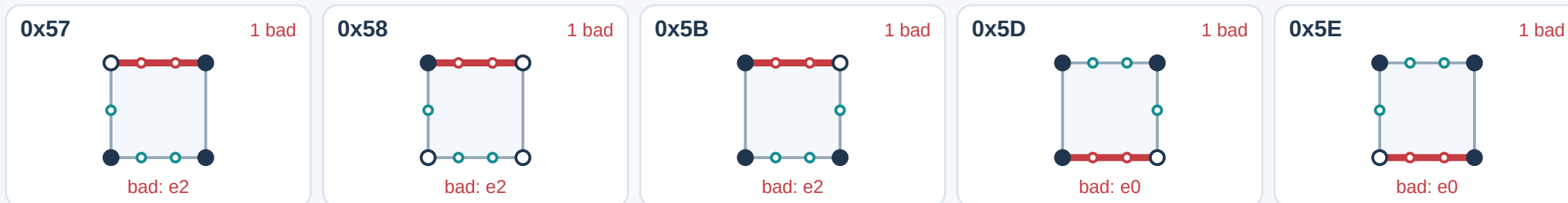
1 条矛盾边 / 本类 104 个 / 第 26-30 个



1 条矛盾边 / 本类 104 个 / 第 31-35 个



1 条矛盾边 / 本类 104 个 / 第 36-40 个

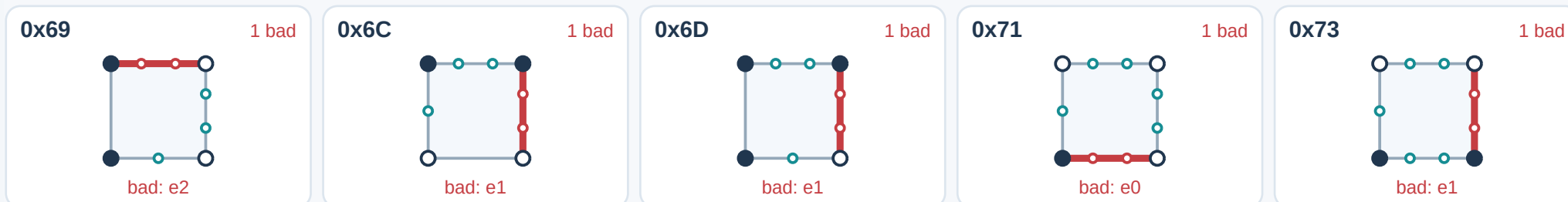


## 非法编码索引 / 1 条矛盾边

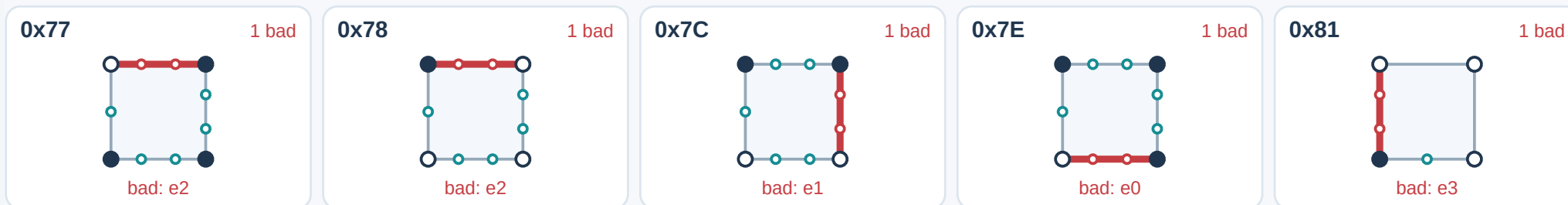
1 条矛盾边 / 本类 104 个 / 第 41-45 个



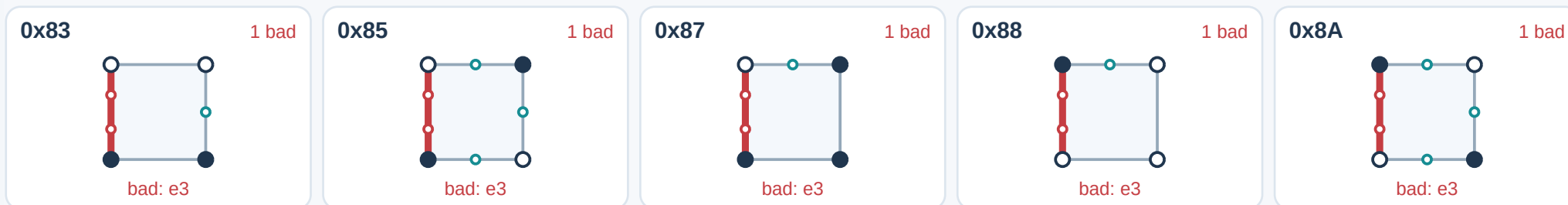
1 条矛盾边 / 本类 104 个 / 第 46-50 个



1 条矛盾边 / 本类 104 个 / 第 51-55 个

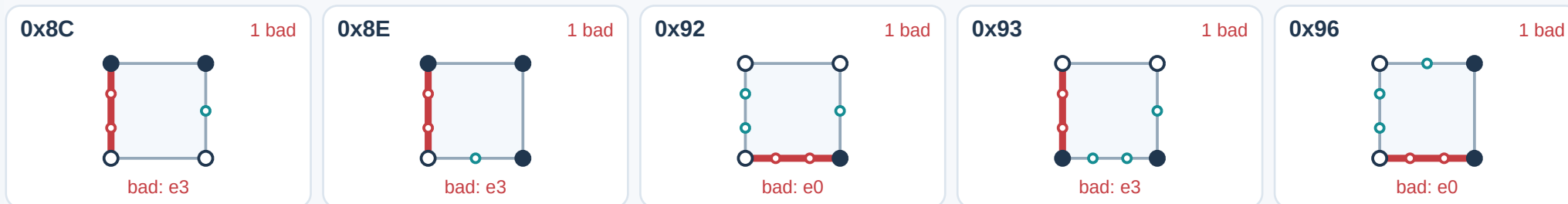


1 条矛盾边 / 本类 104 个 / 第 56-60 个

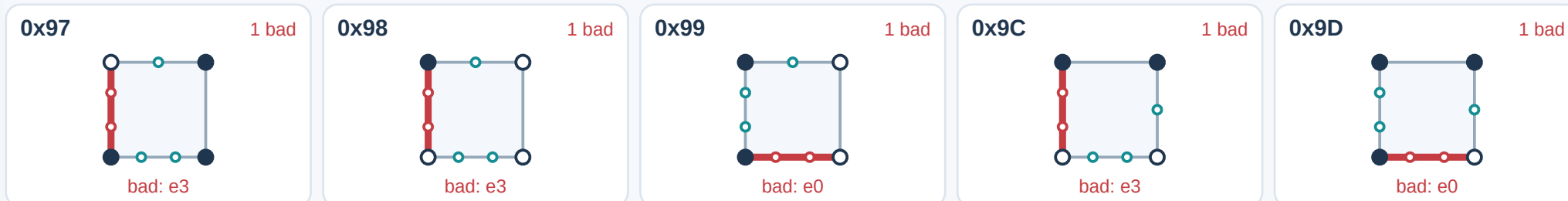


# 非法编码索引 / 1 条矛盾边

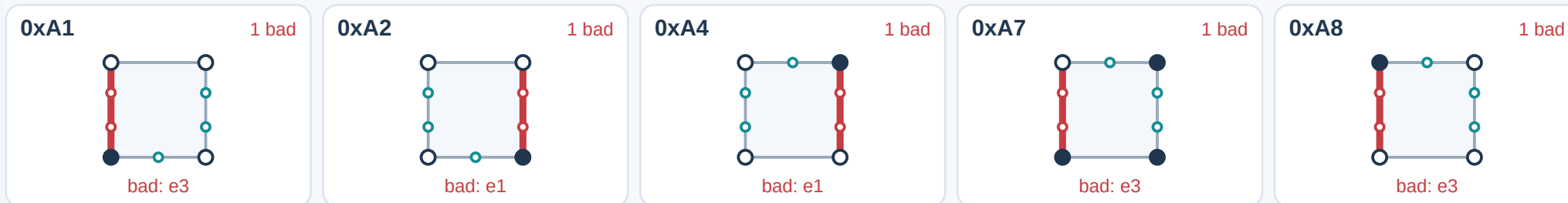
1 条矛盾边 / 本类 104 个 / 第 61-65 个



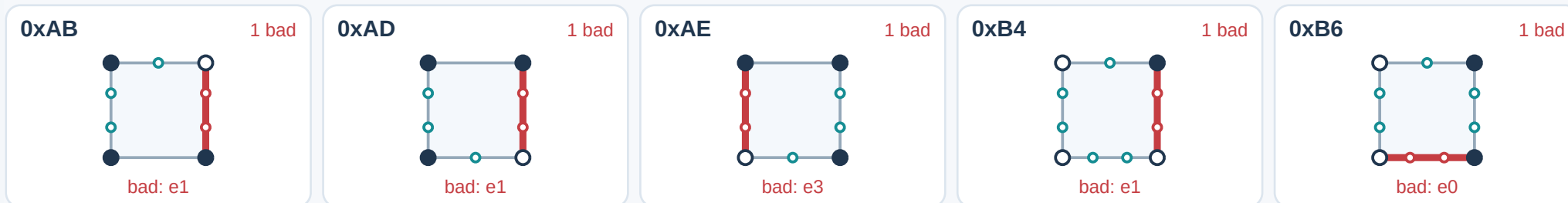
1 条矛盾边 / 本类 104 个 / 第 66-70 个



1 条矛盾边 / 本类 104 个 / 第 71-75 个

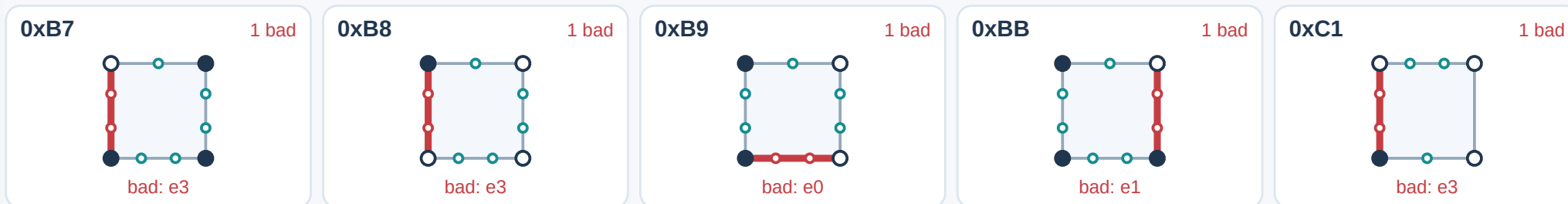


1 条矛盾边 / 本类 104 个 / 第 76-80 个

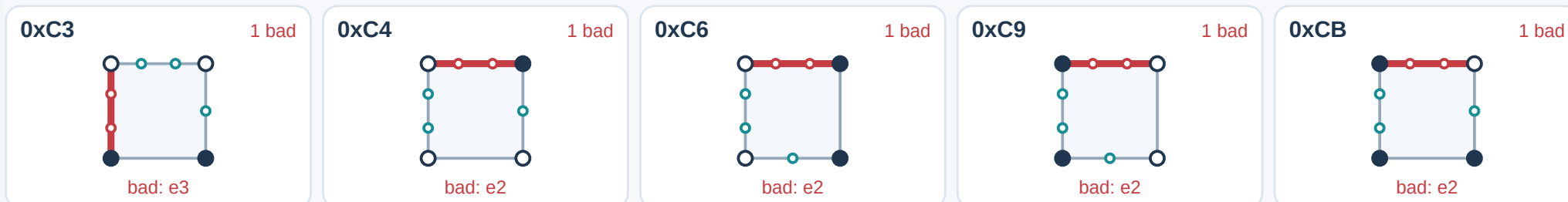


# 非法编码索引 / 1 条矛盾边

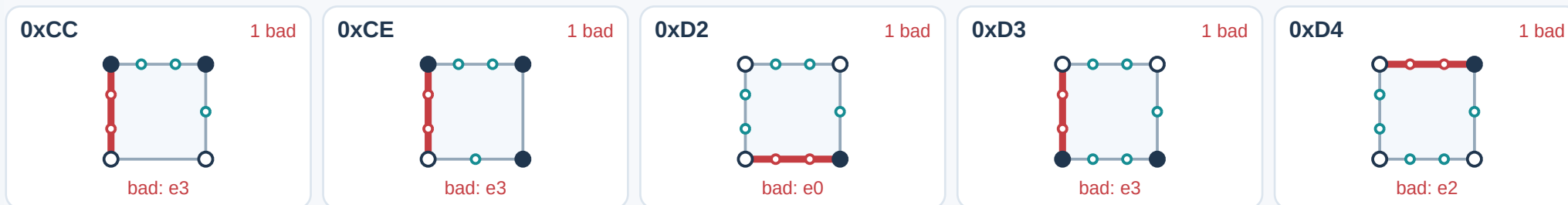
1 条矛盾边 / 本类 104 个 / 第 81-85 个



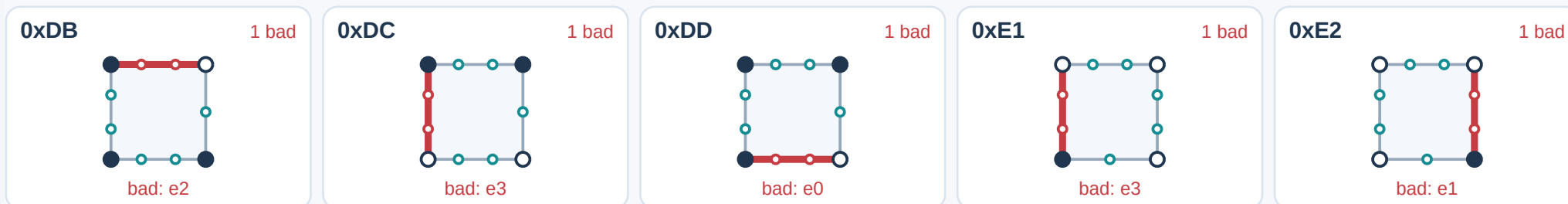
1 条矛盾边 / 本类 104 个 / 第 86-90 个



1 条矛盾边 / 本类 104 个 / 第 91-95 个



1 条矛盾边 / 本类 104 个 / 第 96-100 个



## 非法编码索引 / 1、2 条矛盾边

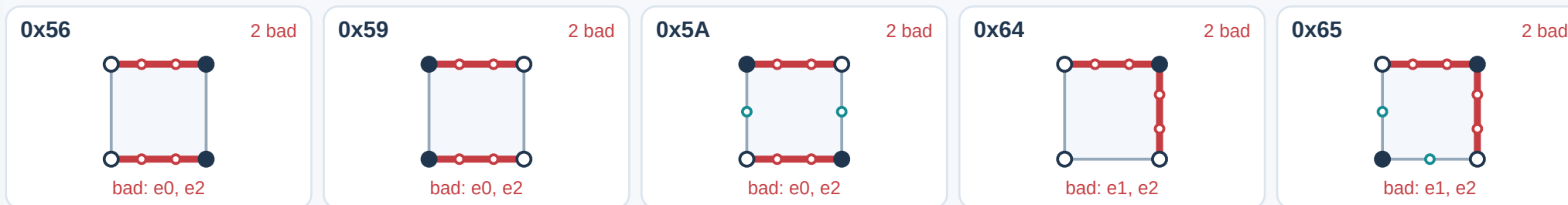
1 条矛盾边 / 本类 104 个 / 第 101-104 个



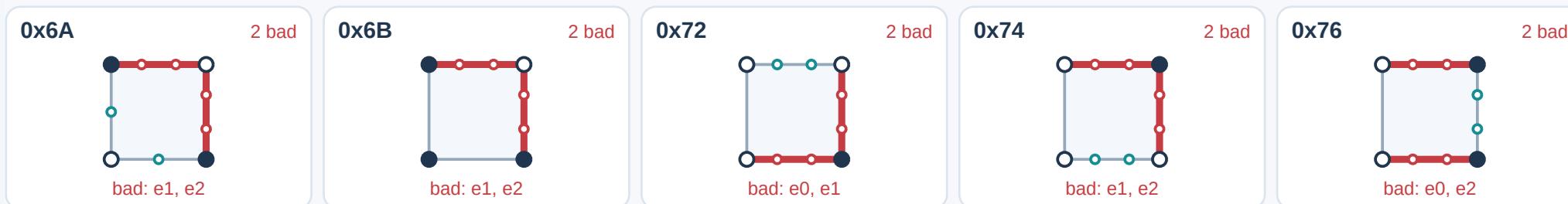
2 条矛盾边 / 本类 60 个 / 第 1-5 个



2 条矛盾边 / 本类 60 个 / 第 6-10 个

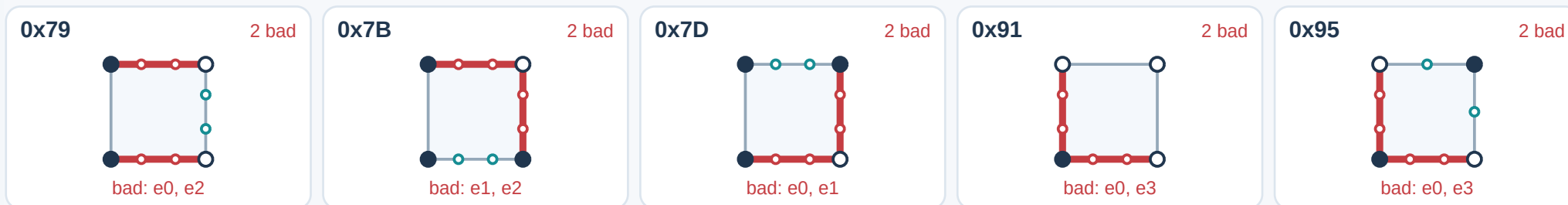


2 条矛盾边 / 本类 60 个 / 第 11-15 个

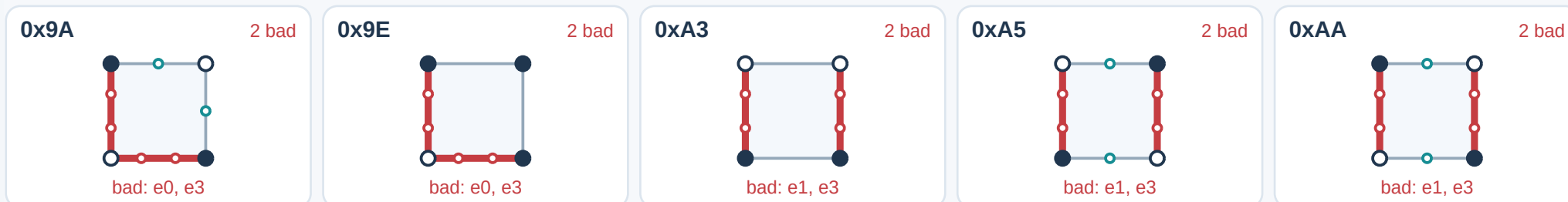


## 非法编码索引 / 2 条矛盾边

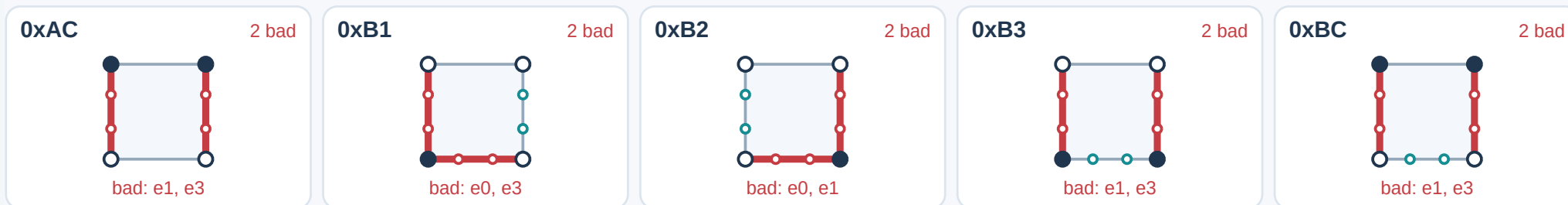
2 条矛盾边 / 本类 60 个 / 第 16-20 个



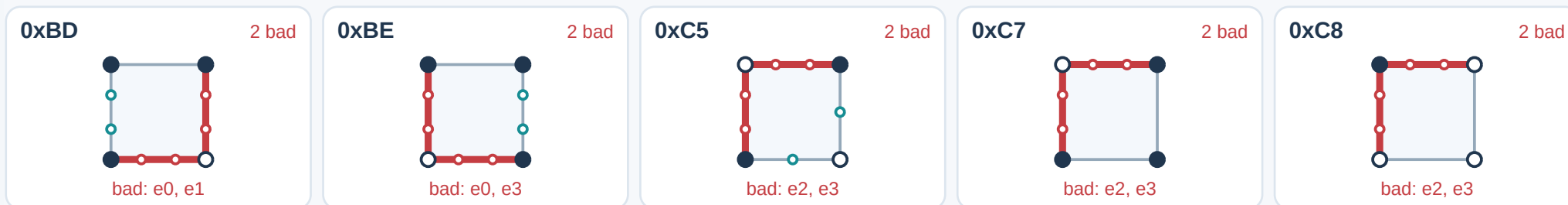
2 条矛盾边 / 本类 60 个 / 第 21-25 个



2 条矛盾边 / 本类 60 个 / 第 26-30 个

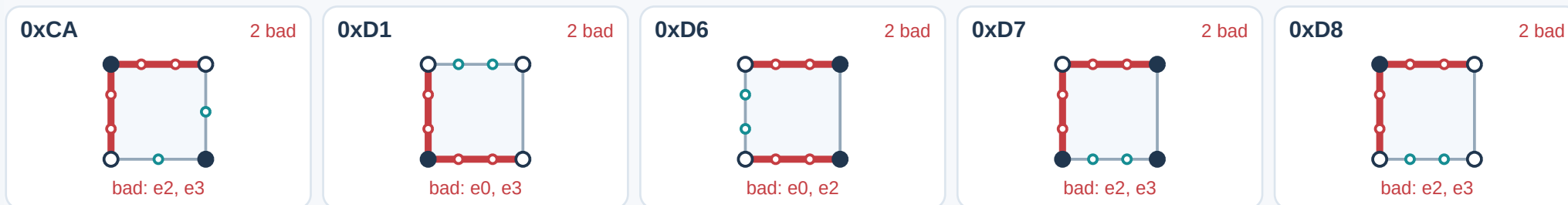


2 条矛盾边 / 本类 60 个 / 第 31-35 个

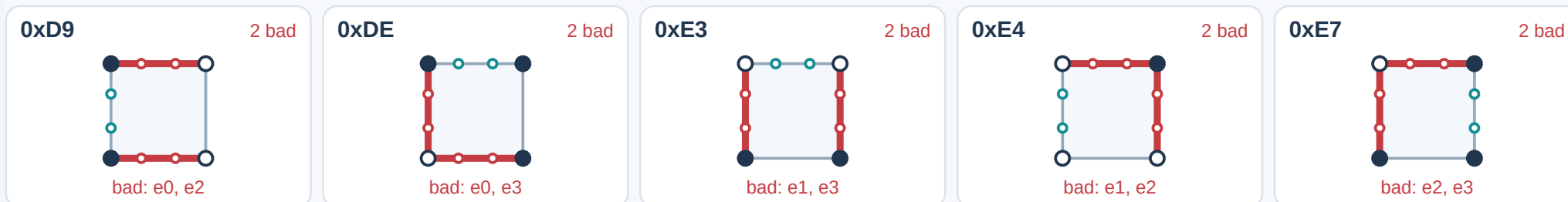


## 非法编码索引 / 2 条矛盾边

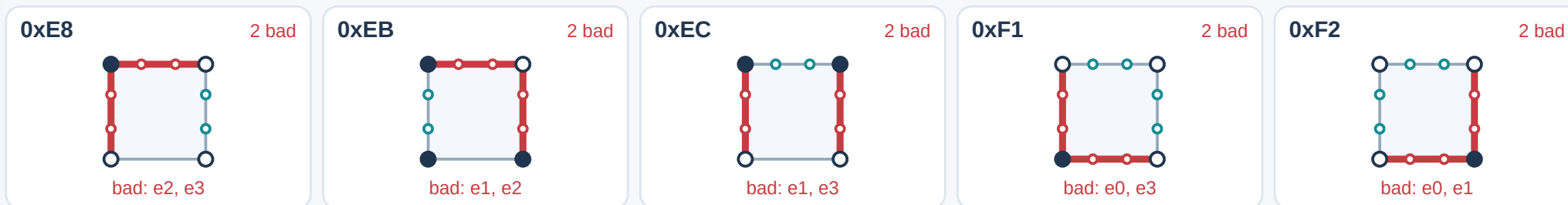
2 条矛盾边 / 本类 60 个 / 第 36-40 个



2 条矛盾边 / 本类 60 个 / 第 41-45 个



2 条矛盾边 / 本类 60 个 / 第 46-50 个

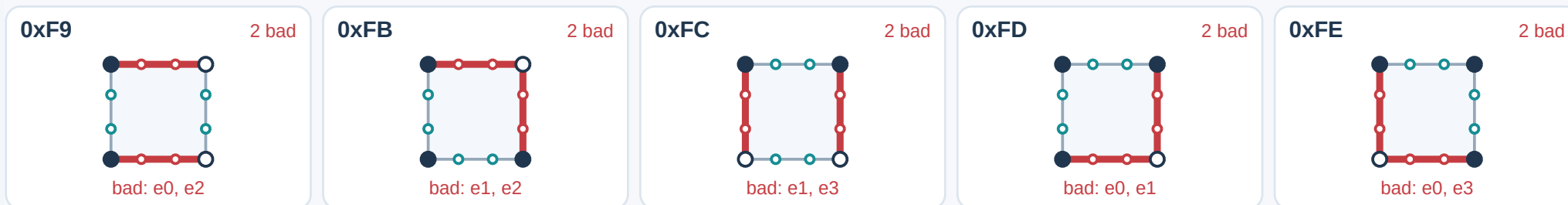


2 条矛盾边 / 本类 60 个 / 第 51-55 个

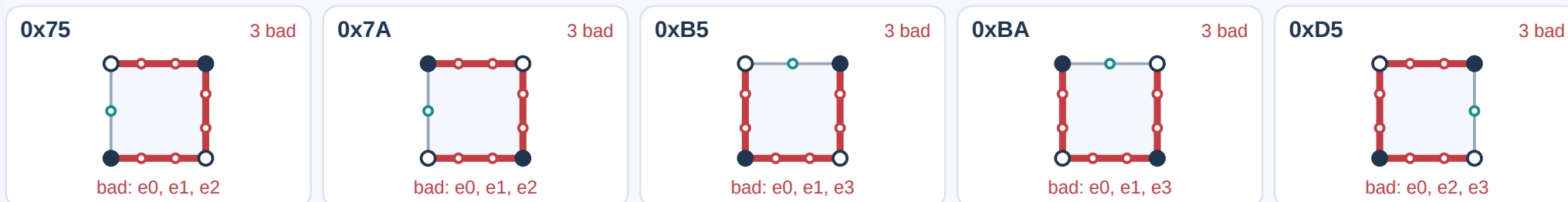


# 非法编码索引 / 2、3、4 条矛盾边

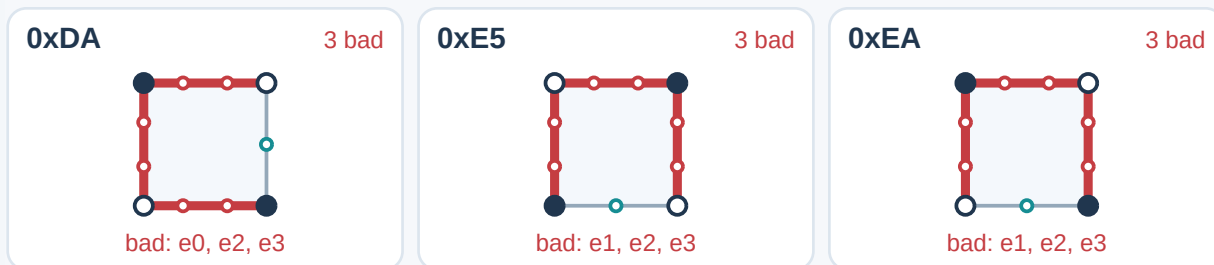
2 条矛盾边 / 本类 60 个 / 第 56-60 个



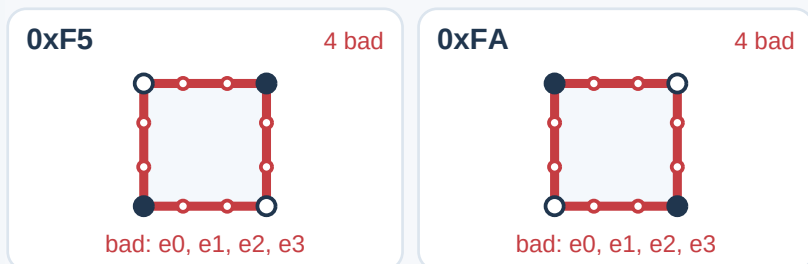
3 条矛盾边 / 本类 8 个 / 第 1-5 个



3 条矛盾边 / 本类 8 个 / 第 6-8 个



4 条矛盾边 / 本类 2 个 / 第 1-2 个

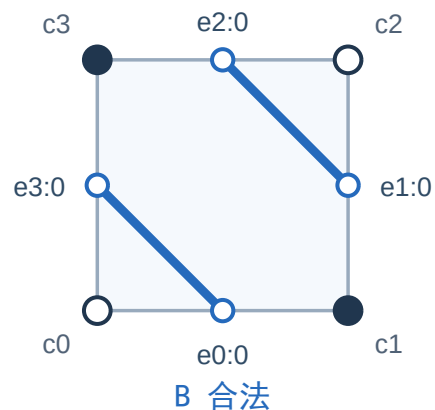
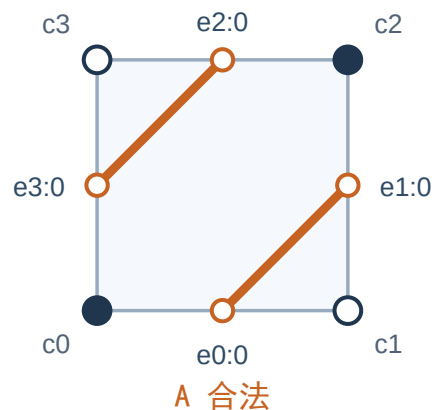


## 附录 / 内外翻转后的连接差异

固定边交点位置，仅翻转所有角点的内外位。配对可能改变，但两侧都满足穿越次数规则。

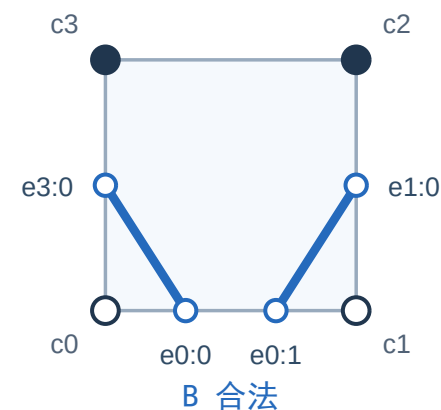
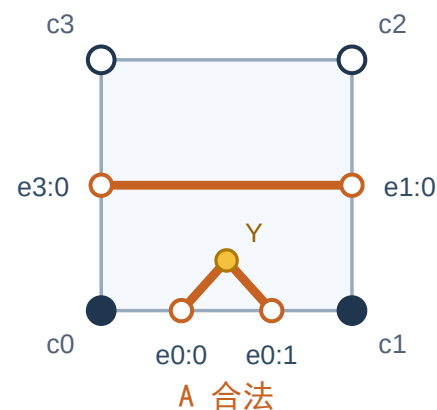
0x05 / 0x0A

4a / 4a



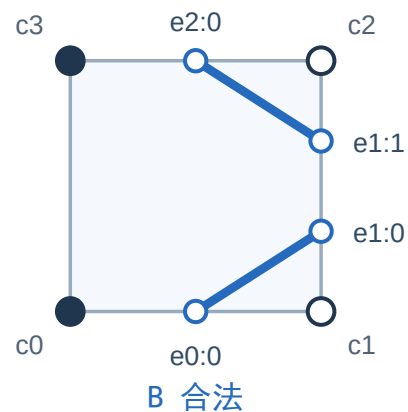
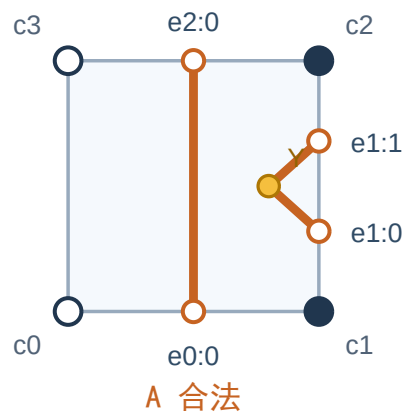
0x13 / 0x1C

3b / 3c



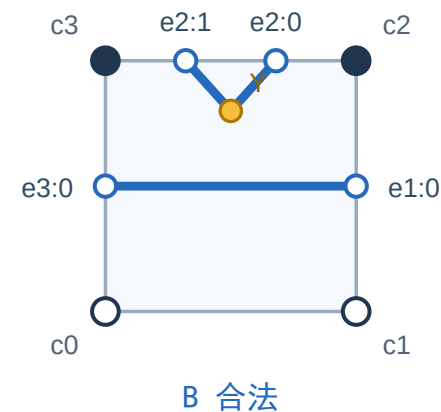
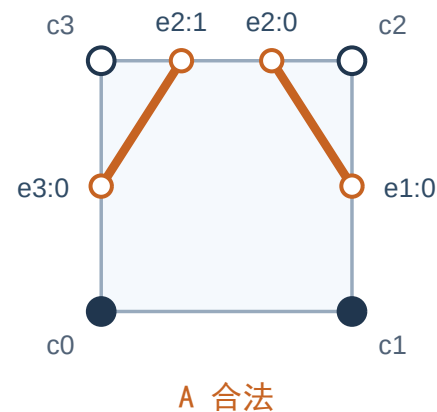
0x26 / 0x29

3b / 3c



0x43 / 0x4C

3c / 3b

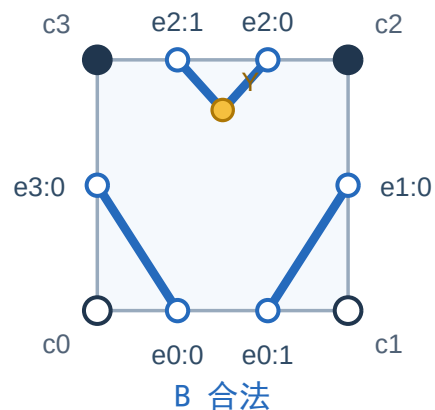
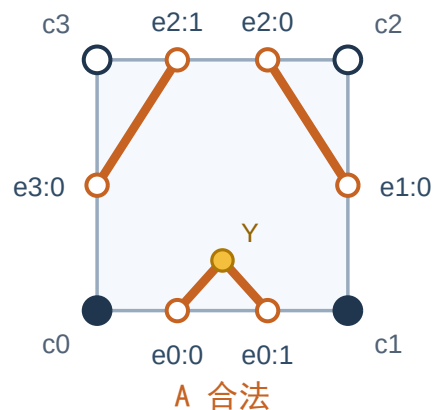


# 附录 / 内外翻转后的连接差异

固定边交点位置，仅翻转所有角点的内外位。配对可能改变，但两侧都满足穿越次数规则。

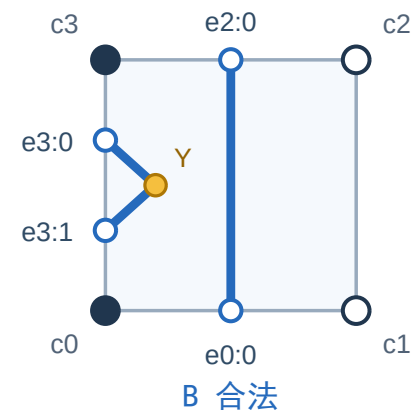
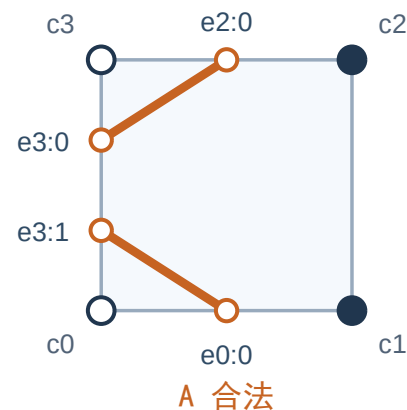
## 0x53 / 0x5C

3d / 3d



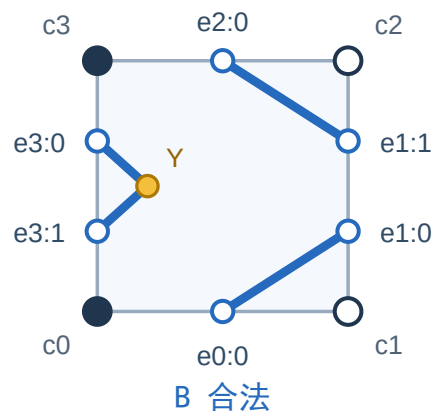
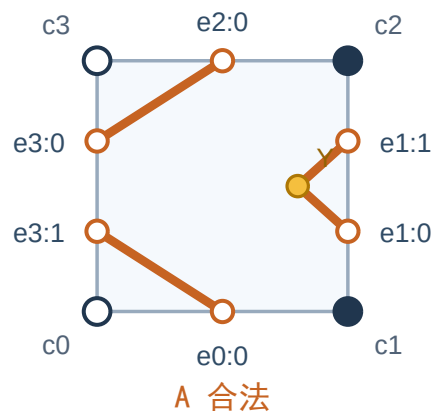
## 0x86 / 0x89

3c / 3b



## 0xA6 / 0xA9

3d / 3d



## 与非法输入的区别

这些是两个不同的合法输入，各自有确定的查表结果。174 个非法编码则连端点符号与交点数都无法同时满足。

黄色点是面内转折点。完整立方体对照、交互图和可单独下载的 SVG 均保留在仓库中。